

```

# =====

# Iris Flower Classification: Logistic Regression vs. Softmax Regression

# =====

# This script demonstrates how to classify the Iris dataset using two variations
# of logistic regression and compares their performance and underlying mechanics.

# =====


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns


# Scikit-learn modules
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix


def main():

    # =====

    # 1. LOAD AND PREPARE THE DATA

    # =====

    print("Loading and preparing the Iris dataset...")

    iris = load_iris()

    X = iris.data    # Features: sepal length, sepal width, petal length, petal width
    y = iris.target  # Target: 0 (Setosa), 1 (Versicolor), 2 (Virginica)

    feature_names = iris.feature_names
    target_names = iris.target_names

```

```

# Split the data into training (80%) and testing (20%) sets

# random_state ensures reproducibility, stratify ensures class balance in splits
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Standardize the features (mean=0, variance=1)
# This is crucial for logistic regression to converge faster and perform optimally
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"Training set shape: {X_train_scaled.shape}")
print(f"Testing set shape: {X_test_scaled.shape}\n")

# =====
# 2. TRAIN LOGISTIC REGRESSION (ONE-VS-REST / OvR)
# =====

print("--- Training One-vs-Rest (Binary Logistic Regression) ---")
# multi_class='ovr' fits one binary classifier per class (e.g., Class A vs. Not Class A)
# solver='lbfgs' is a robust optimizer that supports both 'ovr' and 'multinomial'
log_reg_ovr = LogisticRegression(multi_class='ovr', solver='lbfgs', max_iter=200, random_state=42)
log_reg_ovr.fit(X_train_scaled, y_train)

# Make predictions on the test set
y_pred_ovr = log_reg_ovr.predict(X_test_scaled)

print(f"Accuracy: {accuracy_score(y_test, y_pred_ovr):.4f}")

```

```

print("Classification Report:")
print(classification_report(y_test, y_pred_ovr, target_names=target_names))

# =====
# 3. TRAIN SOFTMAX REGRESSION (MULTINOMIAL)
# =====
print("\n--- Training Softmax Regression (Multinomial) ---")
# multi_class='multinomial' fits a single multiclass model optimizing cross-entropy
# loss across all classes simultaneously using the softmax function.
log_reg_softmax = LogisticRegression(multi_class='multinomial', solver='lbfgs', max_iter=200,
random_state=42)
log_reg_softmax.fit(X_train_scaled, y_train)

# Make predictions on the test set
y_pred_softmax = log_reg_softmax.predict(X_test_scaled)

print(f"Accuracy: {accuracy_score(y_test, y_pred_softmax):.4f}")
print("Classification Report:")
print(classification_report(y_test, y_pred_softmax, target_names=target_names))

# =====
# 4. VISUAL COMPARISON: CONFUSION MATRICES
# =====
print("\nGenerating confusion matrix plots...")
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Plot OvR Confusion Matrix

```

```

cm_ovr = confusion_matrix(y_test, y_pred_ovr)
sns.heatmap(cm_ovr, annot=True, fmt='d', cmap='Blues', ax=axes[0],
            xticklabels=target_names, yticklabels=target_names)
axes[0].set_title('Confusion Matrix: One-vs-Rest (OvR)')
axes[0].set_xlabel('Predicted Label')
axes[0].set_ylabel('True Label')

# Plot Softmax Confusion Matrix
cm_softmax = confusion_matrix(y_test, y_pred_softmax)
sns.heatmap(cm_softmax, annot=True, fmt='d', cmap='Greens', ax=axes[1],
            xticklabels=target_names, yticklabels=target_names)
axes[1].set_title('Confusion Matrix: Softmax (Multinomial)')
axes[1].set_xlabel('Predicted Label')
axes[1].set_ylabel('True Label')

plt.tight_layout()
plt.show()

```

```

# =====
# 5. TEXT-BASED COMPARISON SUMMARY
# =====

print("\n" + "="*70)
print("COMPARISON SUMMARY: One-vs-Rest vs. Softmax Regression")
print("="*70)
print("1. Approach:")
print("  - OvR: Trains K independent binary classifiers (for K classes).")
print("  - Softmax: Trains 1 unified multiclass classifier.")
print("\n2. Probability Output:")
print("  - OvR: Probabilities are calculated independently per class and may")

```

```
print("      not sum exactly to 1.0.")
print(" - Softmax: Probabilities are normalized via the softmax function,")
print("      guaranteeing they sum to exactly 1.0.")
print("\n3. Decision Boundary:")
print(" - OvR: Can be disjointed or overlapping between classes.")
print(" - Softmax: Mutually exclusive, well-defined boundaries between all classes.")
print("\n4. Best Use Case:")
print(" - OvR: When classes are NOT mutually exclusive (multi-label classification).")
print(" - Softmax: When classes ARE strictly mutually exclusive (like the Iris dataset).")
print("="*70)

if __name__ == "__main__":
    main()
```